GitHub Engineering Showcases

Frank Brase

Table of Contents

About this portfolio 1
Vehicle Configurator 2
Engineering Knowledge Assistant 3

About this portfolio

Hands-on software architecture — from embedded systems to cloud-connected and AI-assisted applications — demonstrated through working GitHub showcases.

This document collects selected engineering showcases from my GitHub portfolio:

github.com/fbrase799

The repository is intended to make my engineering knowledge tangible: architecture decisions, implementation trade-offs, and delivery practices are shown through runnable systems rather than slides alone.

Each showcase typically covers:

- problem statement and scope
- architecture and technology choices
- implementation highlights
- testing and documentation
- a live demonstration where available

The profile README and dedicated showcase pages on GitHub are the primary references; this PDF is a compact overview for sharing offline.

Vehicle Configurator

A live, multi-step vehicle configuration application combining a Java/Spring Boot REST backend with a Vue-based interactive frontend.



Architecture & technologies

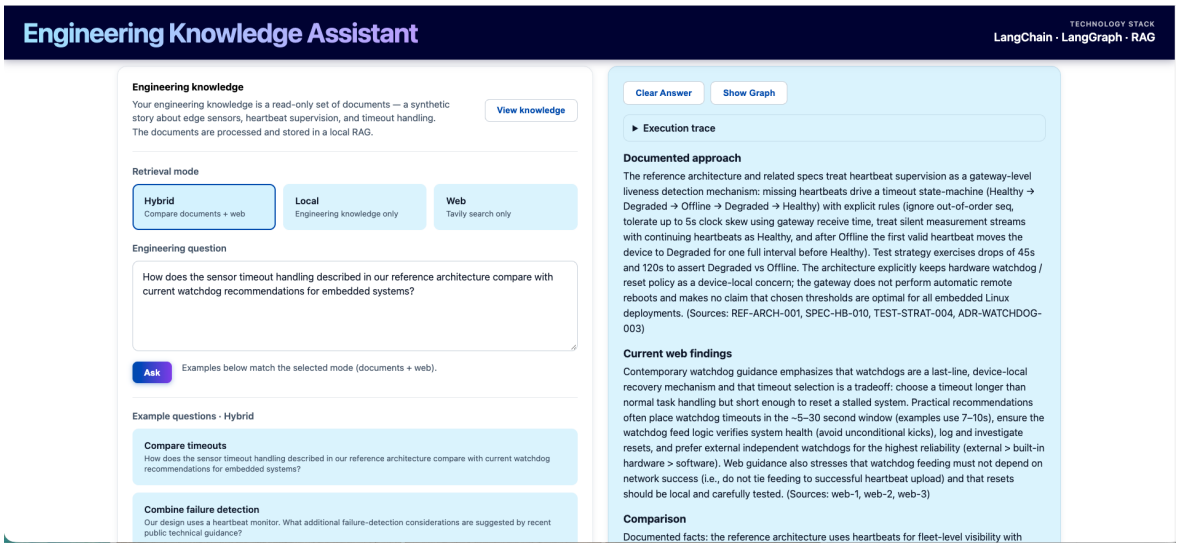
- **Frontend:** Vue 3, Vite, Three.js — interactive 3D vehicle preview and real-time price updates
- **Backend:** Java 25, Spring Boot 4, JPA/Hibernate — REST API, server-side pricing and order submission
- **Persistence:** SQLite embedded in the backend
- **Deployment:** Docker Compose and Azure Container Apps with separate frontend and backend containers
- **Delivery:** GitHub Actions CI/CD with OIDC-based Azure deployment

Engineering focus: Full-stack software architecture, separation of concerns, reproducible container deployment and automated delivery.

Links: Live demonstration | Source code | Technical details

Engineering Knowledge Assistant

An engineering Q&A demonstrator that compares a curated local knowledge base with current public web information. LangGraph orchestrates retrieval, web search and LLM synthesis within explicit execution limits.



Architecture & technologies

- **Frontend:** React, TypeScript, Vite — response view and an SSE-based execution timeline
- **Backend:** FastAPI, LangGraph, LangChain — stateful orchestration with conditional routing
- **Retrieval:** FAISS and FastEmbed MiniLM — shared read-only document corpus
- **Generation:** Tavily web search and Azure OpenAI — bounded search and synthesis calls
- **Deployment:** Docker Compose and Azure Container Apps

Engineering focus: Hybrid RAG and web-based Q&A, traceable execution, explicit request budgets, and source validation.

Links: Live demonstration | Source code | Technical details